

((به نام خدا))

فاطمه پاک سیما(40006593)

داکیومنت پروژه دوم هوش مصنوعی

الگوریتم ژنتیک برای مسئله فروشنده دوره گرد

مقدمه

الگوریتم ژنتیک (Genetic Algorithm) یا (GA) یک الگوریتم بهینه سازی هیوریستیک است که الهام گرفته از فرایند انتخاب طبیعی و تکامل ژنتیک است. در این پروژه، الگوریتم ژنتیک برای حل مسئله فروشنده دوره گرد (Travelling Salesman Problem یا TSP) پیاده سازی شده است که هدف آن پیدا کردن کوتاه ترین مسیری است که تمام شهرها را دیده و به شهر ابتدایی بازمی گردد.

توابع الگوریتم ژنتیک و عملکرد آنها:

1. generate_population(population_size, cityNum)

- ایجاد جمعیت اولیه از مسیرها با حلقه ایجاد و اضافه کردن شهرها به مسیرها به صورت تصادفی.
- **مراحل ایجاد خروجی**:

1. ایجاد یک مسیر خالی.

2. انتخاب یک شهر به عنوان شهر مبدا و اضافه کردن آن به مسیر.

3. برگرداندن جمعیت اولیه شامل مسیرهای تولید شده.

2. fitness(path)

- محاسبه فاصله یک مسیر با حلقه محاسباتی بر اساس فواصل بین شهرها.

- **مراحل ایجاد خروجی**:

1. از ماتریس فواصل بین شهرها، فاصله بین هر دو شهر متوالی گرفته می شود.

2. فواصل به هم جمع می شوند تا مسافت کل مسیر محاسبه شود.

3. برگرداندن مقدار فیتنس مسیر.

3. roulette_wheel_selection(population, fitness_scores)

- انتخاب والدین بر اساس احتمالات محاسبه شده با حلقه تکراری و انتخاب تصادفی.

- **مراحل ایجاد خروجی**:

1. محاسبه احتمال انتخاب هر والد بر اساس فیتنس آنها.

2. انتخاب تصادفی یک عدد و انتخاب والدان متناسب با احتمالات محاسبه شده.

3. برگرداندن والدین انتخاب شده.

4. greedy_selection(population, fitness_scores)

- انتخاب والدین با کمترین امتیاز فیتنس برای تولید فرزندان با حلقه تکراری و انتخاب دو والد.

- **مراحل ایجاد خروجی**:

1. انتخاب دو والد با کمترین فیتنس از جمعیت.

2. انتخاب این دو والد برای تولید فرزندان.

3. برگرداندن والدان و فرزندان انتخاب شده.

5. single_point_crossover(parent1, parent2)

- تولید دو فرزند با ترکیب بخش‌های از دو مسیر والدین با حلقه تکراری و انتخاب نقطه تقاطع.

- **مراحل ایجاد خروجی**:

1. انتخاب نقطه تقاطع تصادفی و ترکیب مسیرهای والدین.

2. تولید فرزندان جدید با ترکیب بخش‌های مشترک از والدین.

3 برگرداندن فرزندان تولید شده.

6. two_point_crossover(parent1, parent2)

- تولید دو فرزند با تبادل قطعه‌هایی از دو مسیر والدین با حلقه تکراری و انتخاب دو نقطه تقاطع.

- **مراحل ایجاد خروجی**:

1. انتخاب دو نقطه تقاطع تصادفی و ترکیب دو مسیر والدین.

2. تولید دو فرزند با تعویض قطعه‌های مختلف از مسیر والدین

3. برگرداندن فرزندان تولید شده.

7. single_point_mutation(chromosome, mutation_rate=0.3)

- اعمال جهش نقطه‌ای تک روی یک مسیر با انتخاب یک ژن تصادفی و جابه‌جایی آن با حلقه تکراری.

- **مراحل ایجاد خروجی**:

1. انتخاب یک ژن تصادفی از مسیر.

2. جابه‌جایی ژن انتخابی با یک ژن تصادفی دیگر.

3. برگرداندن مسیر با جهش اعمال شده.

8. two_point_mutation(chromosome, mutation_rate=0.3)

- اعمال جهش دو نقطه‌ای روی یک مسیر با انتخاب دو نقطه تصادفی و تبدیل قطعه‌های مختلف از مسیر با حلقه تکراری.

- **مراحل ایجاد خروجی**:

1. انتخاب دو نقطه تصادفی از مسیر.

2. تبدیل قطعه‌های مختلف مسیر میان دو نقطه انتخابی.

3. برگرداندن مسیر با جهش اعمال شده.

9. `replace_least_fit(population, fitness_scores, offspring1, offspring2)`

- جایگزینی فرزندان تولیدی به جای دو فرد کم‌فیتتر جمعیت با حلقه تکراری و انتخاب دو فرزند.

- **مراحل ایجاد خروجی**:

1. یافتن دو نفر با کمترین امتیاز فیتنس از جمعیت.

2. جایگزینی این دو فرد با فرزندان تولید شده.

3. برگرداندن جمعیت جدید.

10. `genetic_algorithm(population_size, generations)`

- حلقه اصلی اجرای الگوریتم ژنتیک با اجرای گام‌های انتخاب والدین، تراکم، جهش و جایگزینی برای تکامل جمعیت.

- **مراحل ایجاد خروجی**:

1. ایجاد جمعیت اولیه.

2. پیمایش نسل‌ها و اجرای عملیات انتخاب، تراکم، جهش و جایگزینی.

3 برگرداندن بهترین حل و ارزش فیتنس آن.

اجرا

1. تعیین اندازه جمعیت (`population_size`) تعداد نسل‌ها (`generations`) و ماتریس فواصل بین شهرها.

2. تولید جمعیت اولیه با استفاده از `generate_population`

3. محاسبه فیتنس برای هر مسیر با استفاده از `fitness`

4. حلقه برای تکامل نسل‌ها و اجرای عملیات انتخاب، تراکم، جهش و جایگزینی با استفاده از `genetic_algorithm`

5. چاپ بهترین حل و ارزش فیتنس آن پس از تعداد مشخصی از نسل‌ها.

نتیجه‌گیری

این داکيومنت شرح داده‌ای جامع از توابع و عملکرد آنها در الگوریتم ژنتیک برای حل مسئله فروشنده دوره‌گرد است. این الگوریتم از انتخاب، تراکم، جهش و جایگزینی برای بهینه‌سازی مسئله TSP استفاده می‌کند و با پیمایش نسل‌ها به بهترین حل ممکن برای مسئله می‌رسد.

زمان اجرا و دقت :

به ازای 100 بار اجرای برنامه وقتی که $\text{generation} = 10$ ، $\text{population} = 6$

روش چرخ رولت ، ترکیب چند نقطه ای و جهش تک نقطه ای : در 96 درصد مواقع جواب صحیح پیدا شد.

```
96%
Best solution: [0, 2, 1, 3, 0]
Best value: 48

[Done] exited with code=0 in 0.231 seconds
```

روش چرخ رولت ، ترکیب چند نقطه ای و جهش دو نقطه ای : در 97 درصد مواقع جواب صحیح پیدا شد.

```
97%
Best solution: [0, 2, 1, 3, 0]
Best value: 48

[Done] exited with code=0 in 0.249 seconds
```

روش چرخ رولت ، ترکیب تک نقطه ای و جهش تک نقطه ای :

```
96%
Best solution: [0, 3, 1, 2, 0]
Best value: 48

[Done] exited with code=0 in 0.213 seconds
```

روش چرخ رولت ، ترکیب تک نقطه ای و جهش دو نقطه ای :

```
91%
Best solution: [0, 2, 1, 3, 0]
Best value: 48

[Done] exited with code=0 in 0.243 seconds
```

روش حریمانه، ترکیب تک نقطه ای و جهش دو نقطه ای :

```
92%
Best solution: [0, 1, 3, 2, 0]
Best value: 58

[Done] exited with code=0 in 0.269 seconds
```

روش حریمانه، ترکیب تک نقطه ای و جهش تک نقطه ای :

```
97%
Best solution: [0, 2, 1, 3, 0]
Best value: 48

[Done] exited with code=0 in 0.198 seconds
```

روش حریمانه، ترکیب چند نقطه ای و جهش تک نقطه ای :

```
93%
Best solution: [0, 1, 3, 2, 0]
Best value: 58

[Done] exited with code=0 in 0.23 seconds
```

روش حریصانه، ترکیب چند نقطه ای و جهش دو نقطه ای :

```
93%
Best solution: [0, 3, 1, 2, 0]
Best value: 48

[Done] exited with code=0 in 0.332 seconds
```

با تغییر در روش های انتخاب و ترکیب و همچنین جهش مدت زمان رسیدن به پاسخ و درصد یافتن جواب های صحیح در 100 بار اجرای این کد برای ماتریس های ورودی متفاوت می شود اما چون ماتریس ورودی ماتریس کوچکی هست تمامی روش ها در مدت زمان های نزدیک بهم میتوانند به جواب بهینه برسند.

اما اگر ماتریس بزرگتر و پیچیده تری در ورودی داشته باشیم تفاوت این روش ها آشکار تر است.

مثلا برای ماتریس ورودی زیر:

```
matrix = [[0, 2, 9, 10, 7],
          [2, 0, 4, 6, 3],
          [9, 4, 0, 7, 8],
          [10, 6, 7, 0, 5],
          [7, 3, 8, 5, 0]]
```

اگر جمعیت را 20 و نسل را 100 در نظر بگیریم:

روش چرخ رولت ، ترکیب چند نقطه ای و جهش تک نقطه ای :

```
89%
Best solution: [0, 1, 2, 3, 4, 0]
Best value: 25

[Done] exited with code=0 in 0.315 seconds
```

روش چرخ رولت ، ترکیب چند نقطه ای و جهش دو نقطه ای :

```
79%  
Best solution: [0, 4, 3, 2, 1, 0]  
Best value: 25  
  
[Done] exited with code=0 in 0.269 seconds
```

روش چرخ رولت ، ترکیب تک نقطه ای و جهش تک نقطه ای :

```
74%  
Best solution: [0, 1, 2, 3, 4, 0]  
Best value: 25  
  
[Done] exited with code=0 in 0.241 seconds
```

روش چرخ رولت ، ترکیب تک نقطه ای و جهش دو نقطه ای :

```
85%  
Best solution: [0, 4, 3, 2, 1, 0]  
Best value: 25  
  
[Done] exited with code=0 in 0.257 seconds
```

روش حریصانه، ترکیب تک نقطه ای و جهش دو نقطه ای :

```
91%  
Best solution: [0, 1, 2, 3, 4, 0]  
Best value: 25  
  
[Done] exited with code=0 in 0.217 seconds
```

روش حریصانه، ترکیب تک نقطه ای و جهش تک نقطه ای :

```
86%  
Best solution: [0, 1, 2, 3, 4, 0]  
Best value: 25  
  
[Done] exited with code=0 in 0.199 seconds
```

روش حریصانه، ترکیب چند نقطه ای و جهش تک نقطه ای :

```
85%  
Best solution: [0, 1, 2, 3, 4, 0]  
Best value: 25  
[Done] exited with code=0 in 0.254 seconds
```

روش حریصانه، ترکیب چند نقطه ای و جهش دو نقطه ای :

```
86%  
Best solution: [0, 4, 3, 2, 1, 0]  
Best value: 25  
[Done] exited with code=0 in 0.226 seconds
```

به طور کلی روش حریصانه نسبت به چرخ رولت در درصد بالاتری از موارد و با زمان بهتری به جواب بهینه میرسد.